



GATEWAY TECHNICAL

Foundations of AI, ML, and Data Science

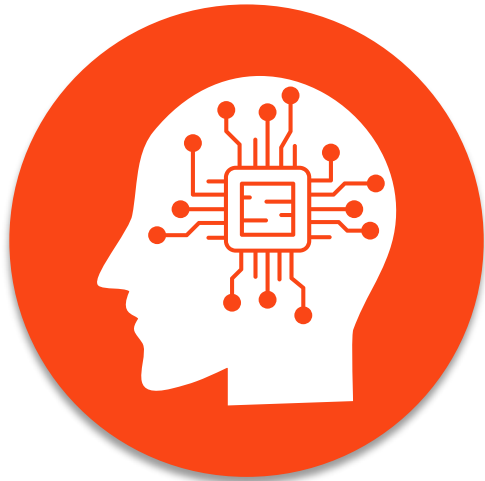


UNIVERSITY OF FLORIDA



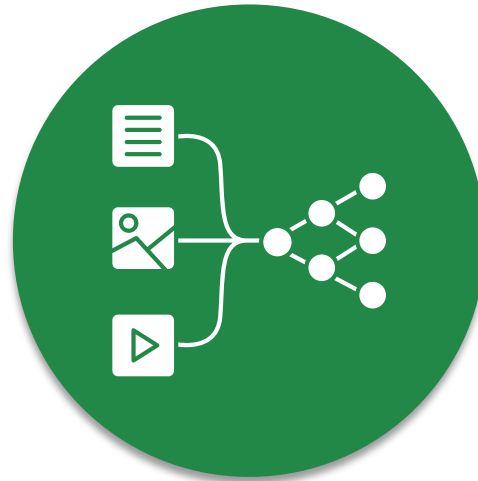
1. Defining the Field: AI, Machine Learning, and Deep Learning

Artificial Intelligence is the broad field, while Machine Learning and Deep Learning are subsets



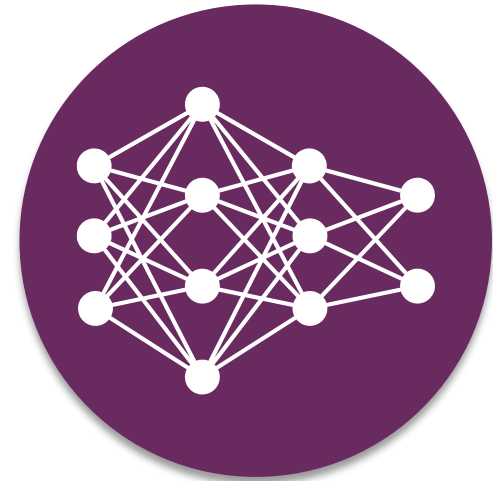
Artificial Intelligence (AI)

Systems performing tasks requiring human-like intelligence (reasoning, planning, perception, problem-solving)



Machine Learning (ML)

Subfield of AI where systems improve through data-driven learning instead of explicit programming



Deep Learning (DL)

Subset of ML using deep neural networks to learn hierarchical representations

The relationship between AI, ML, and DL can be shown as a set of nested categories

- AI includes both symbolic systems (like expert systems) and data-driven systems (like ML).
- Machine Learning focuses on algorithms that adapt to data.
- Deep Learning dominates recent breakthroughs due to scalability with large data and GPUs.

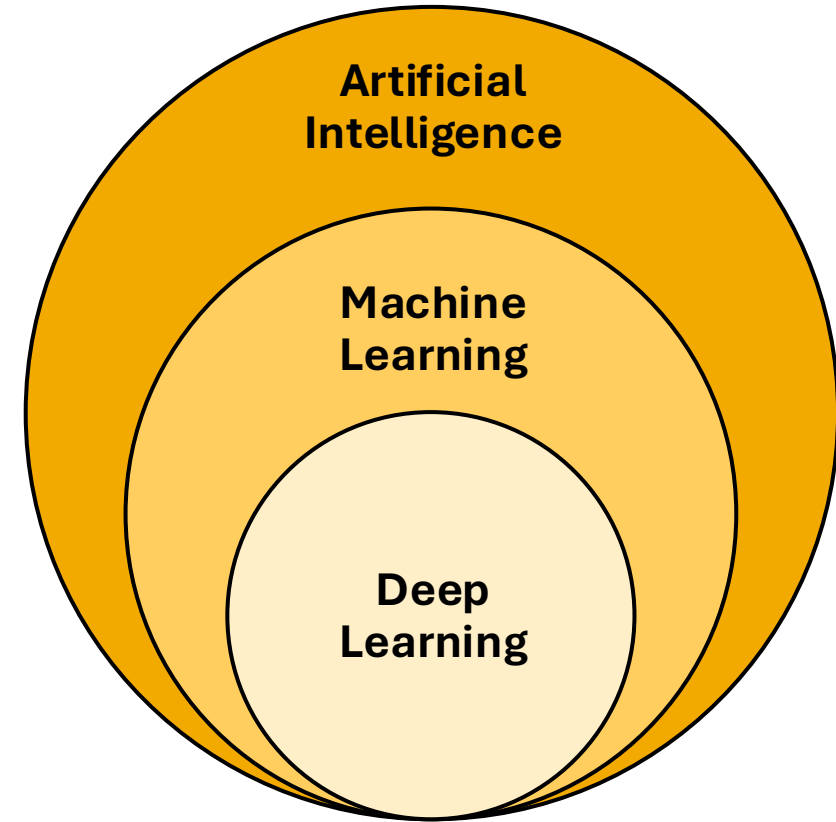


Figure: $AI \supset ML \supset DL$, there are other ways to think of this, but this concept keeps it tidy for us.

Different eras, AI (rules-based): 1980s expert systems with hand-coded rules

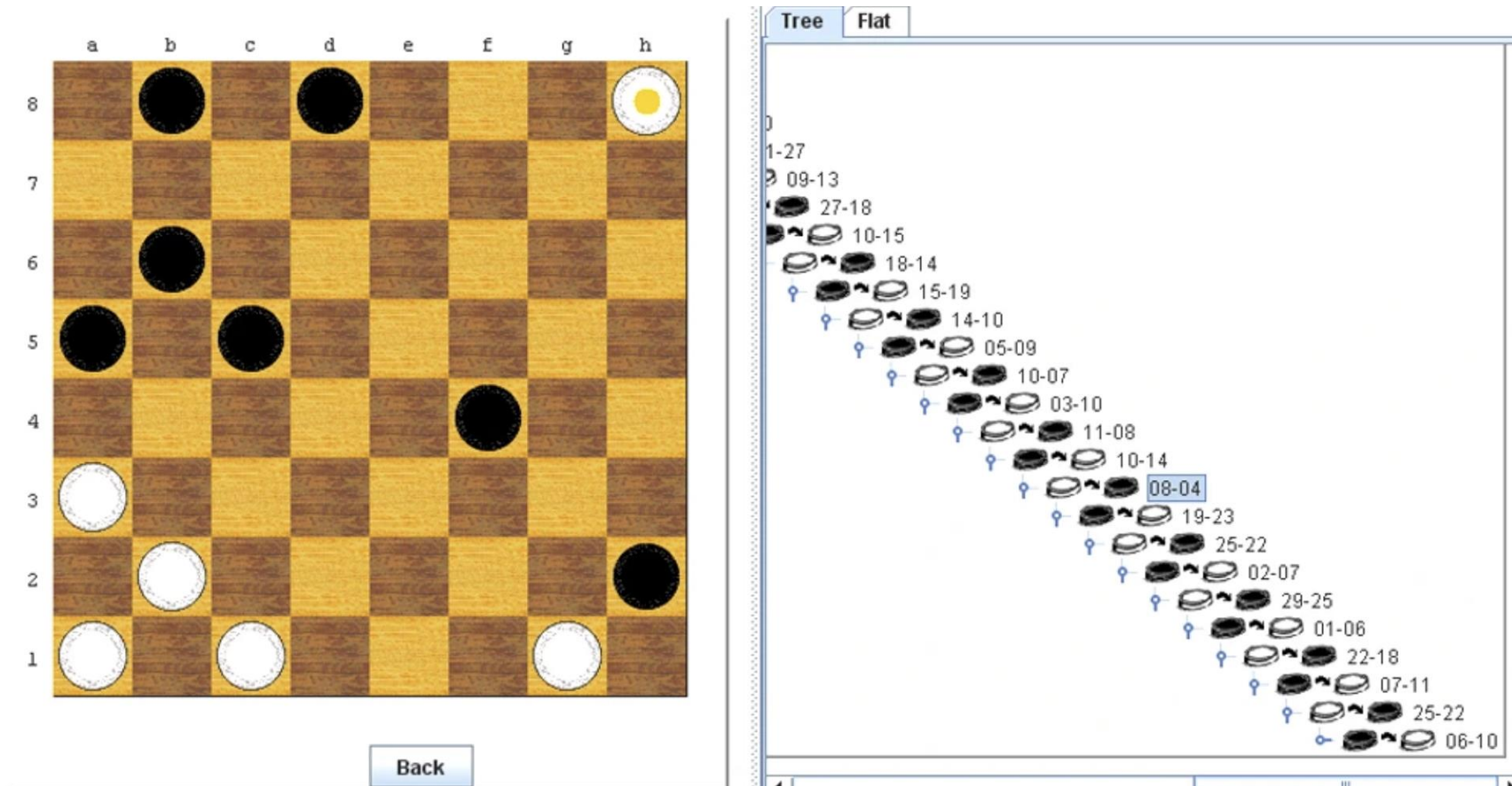


Figure: (Nelson, 2007)

Different eras. Machine Learning: Spam filters, recommendation systems



Different eras. Deep Learning:
Image recognition with CNNs, language models like GPT



Rules-based systems rely on explicit logic, while data-driven systems learn from examples

Rules-based systems

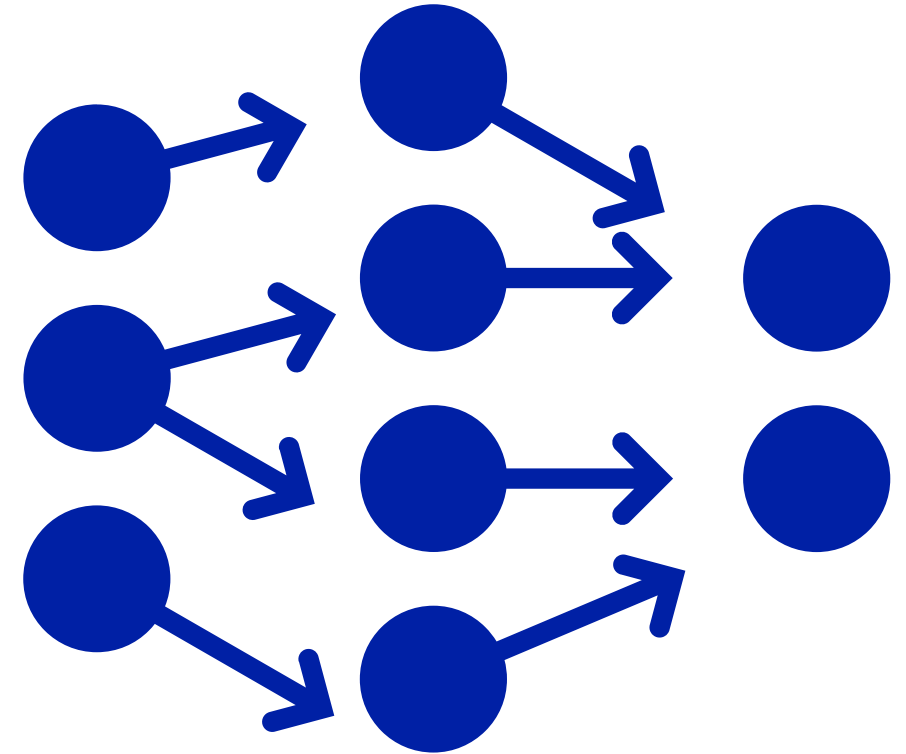
- Hand-crafted if-then logic
- Good for narrow, stable tasks
- Poor at handling complexity or adapting

Data-driven systems

- Learn directly from data
- More flexible and generalizable
- Require large, high-quality datasets

Deep Learning overtook classical ML because it automated feature engineering

- Traditional ML required heavy manual feature engineering.
- DL automates this process by learning hierarchical features directly from raw data.
- With enough data and compute, DL consistently outperforms classical ML.
- This result? Huge gains/progress in computer vision, speech, and NLP.





2. Categories of Machine Learning

Supervised learning uses labeled data to predict outcomes

Definition	The model learns from labeled examples ($X \rightarrow y$).
Goal	Predict an output given an input
Types	Regression: Predict a continuous value • Example: House price prediction from features Classification: Predict a category • Example: Email spam detection (spam vs. not spam)
Key Idea	The model is “supervised” by labels during training.

Unsupervised learning finds hidden structure without labeled data

Definition	The model discovers structure in unlabeled data.
Goal	Group similar items, reduce complexity, and find patterns
Types	Clustering: Customer segmentation in marketing Dimensionality reduction: PCA for visualizing high-dimensional data
Key Idea	Patterns must be inferred — no “correct” answers are given.

Figure: Example of clustering with distinct groups

Reinforcement learning teaches agents to act through trial and error



Definition	An agent learns by interacting with an environment and receiving rewards or penalties.
Key Elements	Agent: Decision-maker Environment: System it interacts with Actions: Choices taken by the agent Rewards: Feedback signal, immediate or delayed
Key Idea	Learn a policy that maximizes cumulative reward

Figure: Reinforcement learning loop of agent, environment, actions, and rewards

Emerging paradigms like self-supervised and few-shot learning are reshaping the field

Self-supervised learning

- Uses raw data to generate labels automatically
- **Example:** Predicting missing words in a sentence
- Foundation of modern language models like BERT and GPT

Few-shot learning

- Models generalize from only a handful of labeled examples
- Enabled by large pretrained models and in-context learning

Different categories of machine learning are best suited to different problems



Supervised regression
Predict sales or prices



Unsupervised clustering
Group shoppers by behavior



Reinforcement learning
Optimize ad placement strategies over time

You should ask:

“Which paradigm fits this problem?”



3. Core Concepts in the Machine Learning Pipeline

A learning task begins by defining inputs and outputs

Problem framing

Identify what the input (X) and output (y) are.

Examples



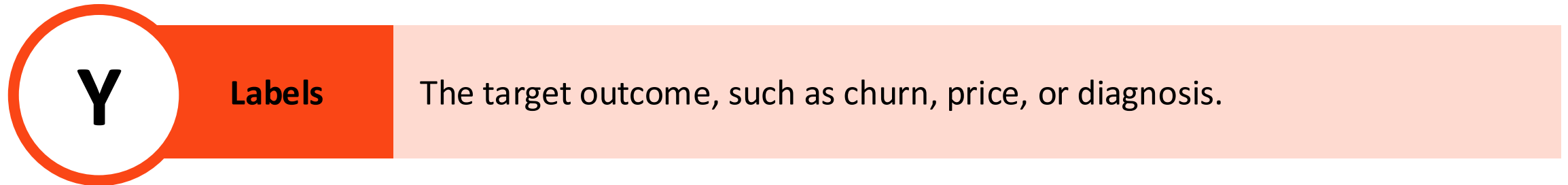
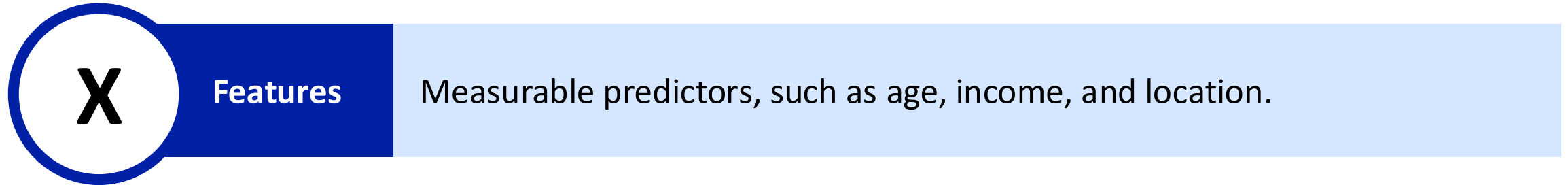
Customer attributes →
Churn (yes/no)



House features →
Price

Not every problem is suitable for ML. Best used for predicting patterns from data.

Machine learning datasets consist of features and labels



Typically organized in tabular form: **rows = instances, columns = variables.**

Splitting data into train, validation, and test sets prevents bias



Purpose	Estimate how well a model generalizes.
Typical split	70–80% training, 20–30% testing.
Validation set	Used to tune hyperparameters or use cross-validation
Analogy	• Training = studying for a test • Validation = practice questions • Test = the real exam

Figure: Illustration of data split into training, validation, and testing sets

Overfitting occurs when a model memorizes data instead of generalizing

Overfitting

Model performs extremely well on training data but poorly on new data

Example: Decision tree that perfectly classifies training data but fails on test data

Generalization

Model captures patterns that apply beyond training

Example: Linear regression that captures broader trends

An end-to-end ML pipeline connects problem definition to deployment

- 1 Problem definition
- 2 Data collection & preprocessing
- 3 Feature engineering and selection
- 4 Model training
- 5 Validation and evaluation
- 6 Deployment into real-world systems
- 7 Monitoring and updating as data evolves

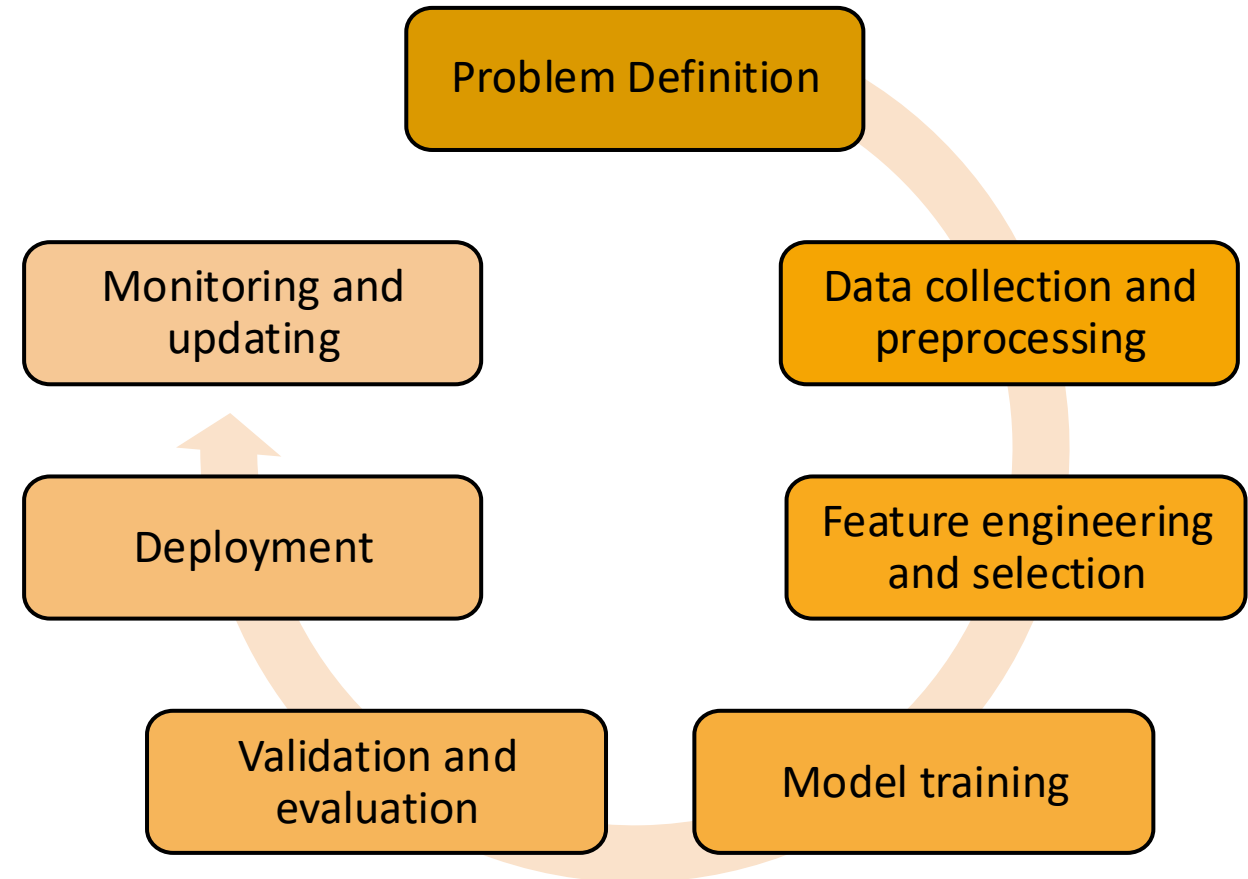


Figure: End-to-end ML pipeline flow

Success in ML depends on the pipeline, not just the model

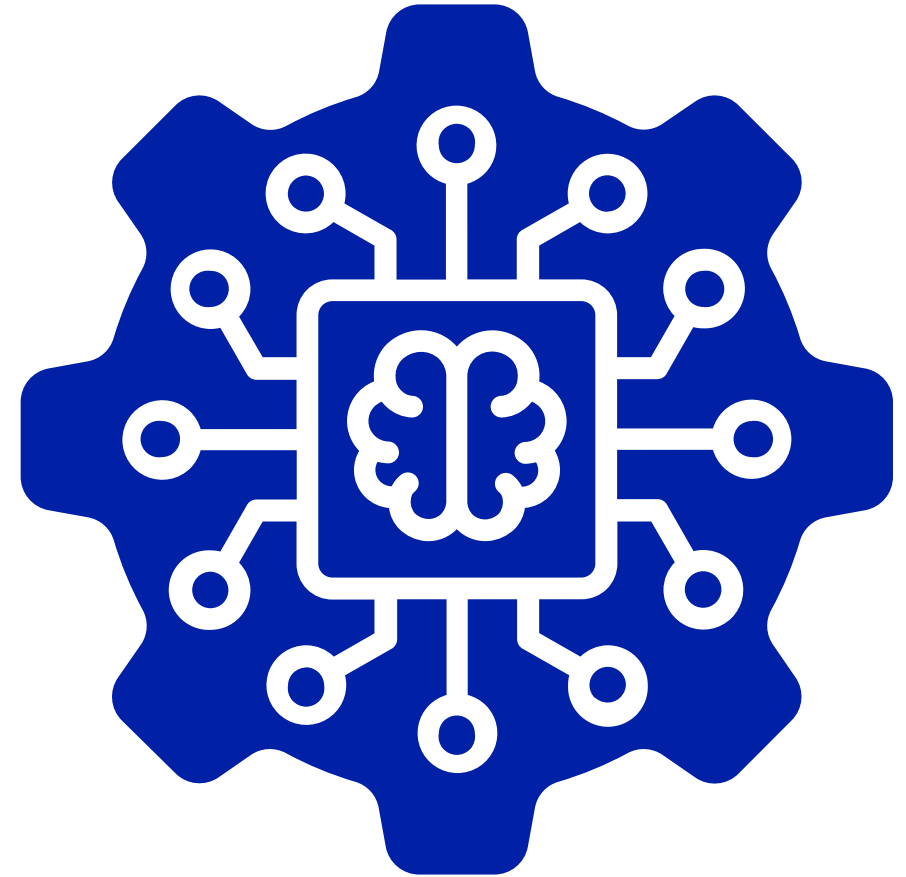
- ML involves the **entire process**, not just algorithms.
- Beginners often spend 90% of their time on model tweaking.
- Just as if not more important:

Problem framing

Data preparation

Careful evaluation

Ongoing monitoring





4. Introduction to Data Preprocessing

Data preprocessing is essential because real-world data is messy

- Models perform only as well as their inputs.
- **“Garbage in, garbage out”:**
Uncleaned data leads to unreliable models.
 - *Is this true? Thoughts....*
- Preprocessing ensures consistency, interpretability, and readiness for modeling.



The rabbit hole of data... stick with me...

Proxies are unavoidable

- Things that are measured are things that CAN BE measured

But many real objectives are:

- delayed
- multi-dimensional
- contested
- difficult to quantify
- Expensive to encode
- Encoded for one purpose used for another

AI systems (and people) **demand objective functions** , so proxies get hard encoded.



Raw data often contains missing values, inconsistencies, and outliers

Problem framing	
Example	Age column with blanks.
Problem	Models cannot directly handle “NA.”
Inconsistent formats	
Example	Dates as strings vs. timestamps; “Male/Female” vs. “M/F.”
Problem	Encoding becomes confused by different representations.
Outliers	
Example	Income recorded as \$1,000,000,000
Problem	Distorts scales, regressions, clustering

	A	B	C	D	E
1	ID	Age	Gender	JoinDate	Income
2	1	25	Male	2020-01-15	50000
3	2		F	15/02/2020	60000
4	3	40	Female	2020/03/20	55000
5	4	35	M	2020-04-25	58000
6	5		male	April 30, 2020	1E+09
7	6	28	F	2020-06-05	62000

Figure: Examples of messy data entries

Tools for handling missing values, encoding categories, and scaling features

Missing values

- Drop rows/columns with excessive missingness.
- Impute with mean, median, mode, or domain-specific rules.
- Add a “missingness indicator” feature.

Categorical variables

- **One-hot encoding:** Convert categories into binary columns.
- **Ordinal encoding:** Map ordered categories to integers.

Scaling and normalization

- **Standardization:** Mean = 0, standard deviation = 1
- **Normalization:** Rescale to range (e.g., 0–1).
- Essential for scale-sensitive models like KNN and neural networks

Data leakage occurs when preprocessing steps improperly use test data



Figure: Remember to fit preprocessing steps on the **training set only**.



5. Feature Engineering

Feature engineering transforms raw data into more useful inputs



Raw Datasets

Raw datasets rarely contain features in the most useful form.

Feature Engineering

Transforming raw variables into informative predictors

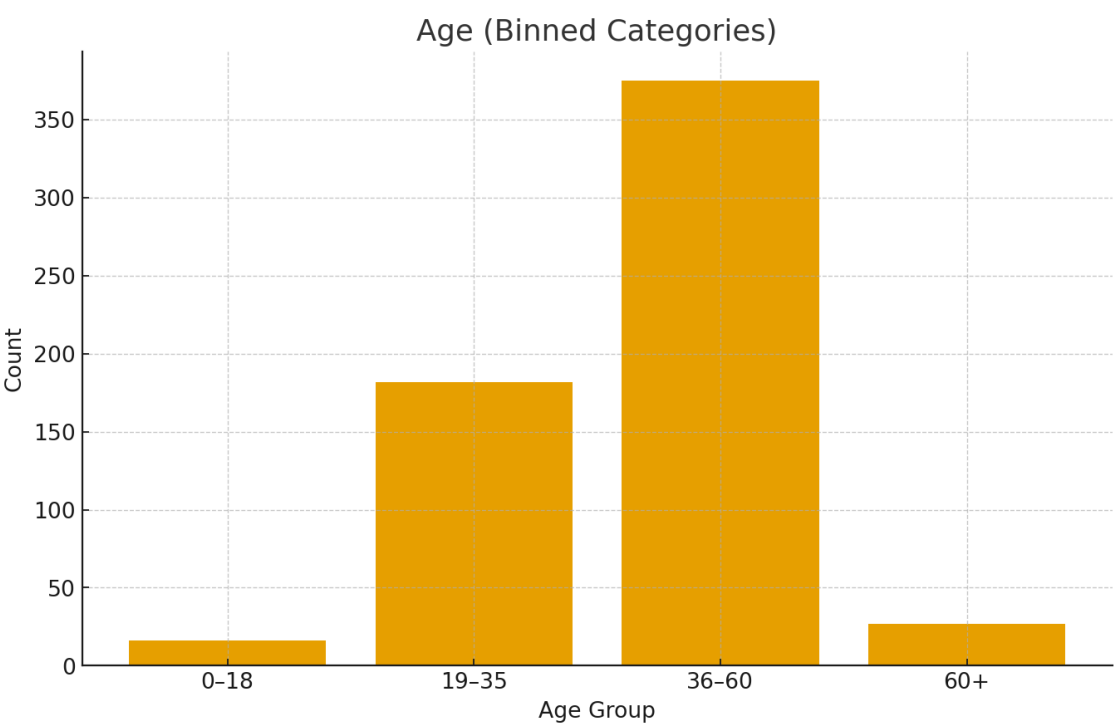
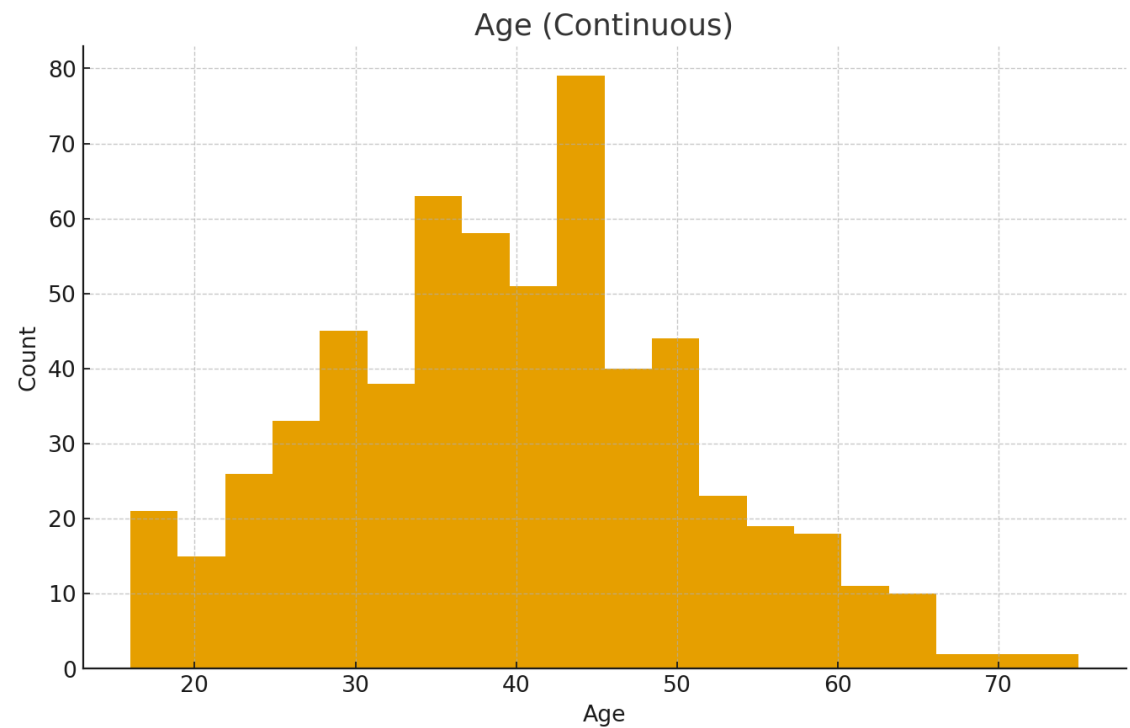
Goal

Improve performance by embedding domain knowledge into data

Example

From “date,” create “day of week,” “is holiday,” or “month.”

Binning: Convert continuous variables into categories.
Example: Age → groups (0–18, 19–35, 36–60, 60+).



Log-Scaling Reduce skewness of variables like income or housing prices.

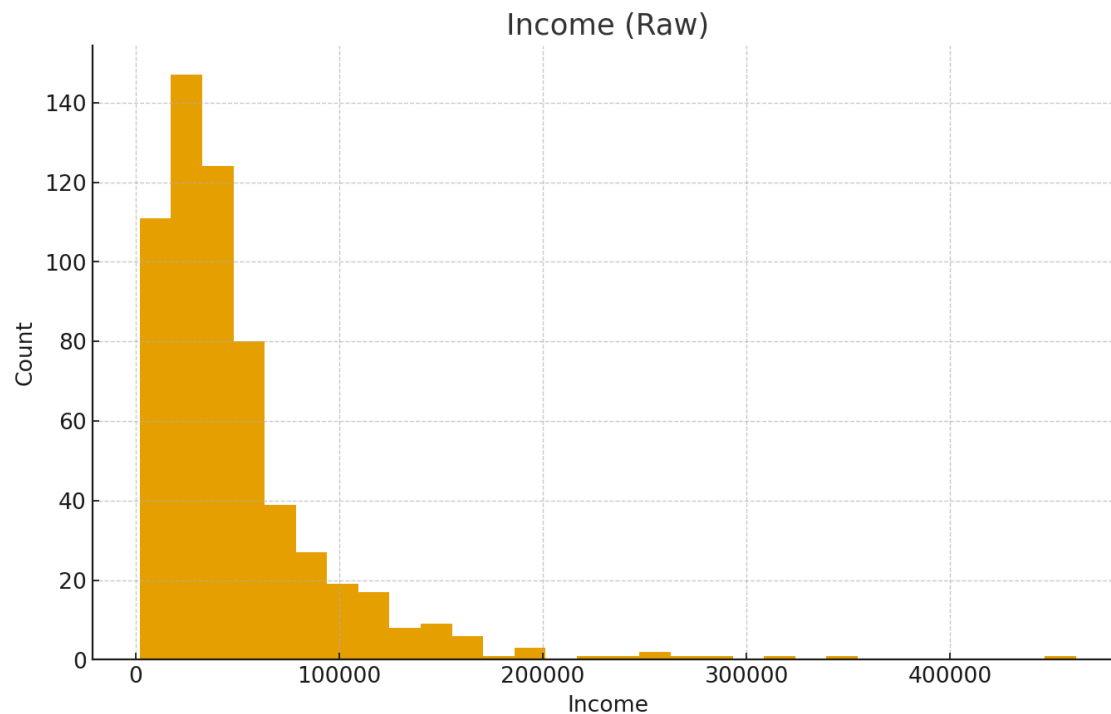


Figure: Example of raw skewed data

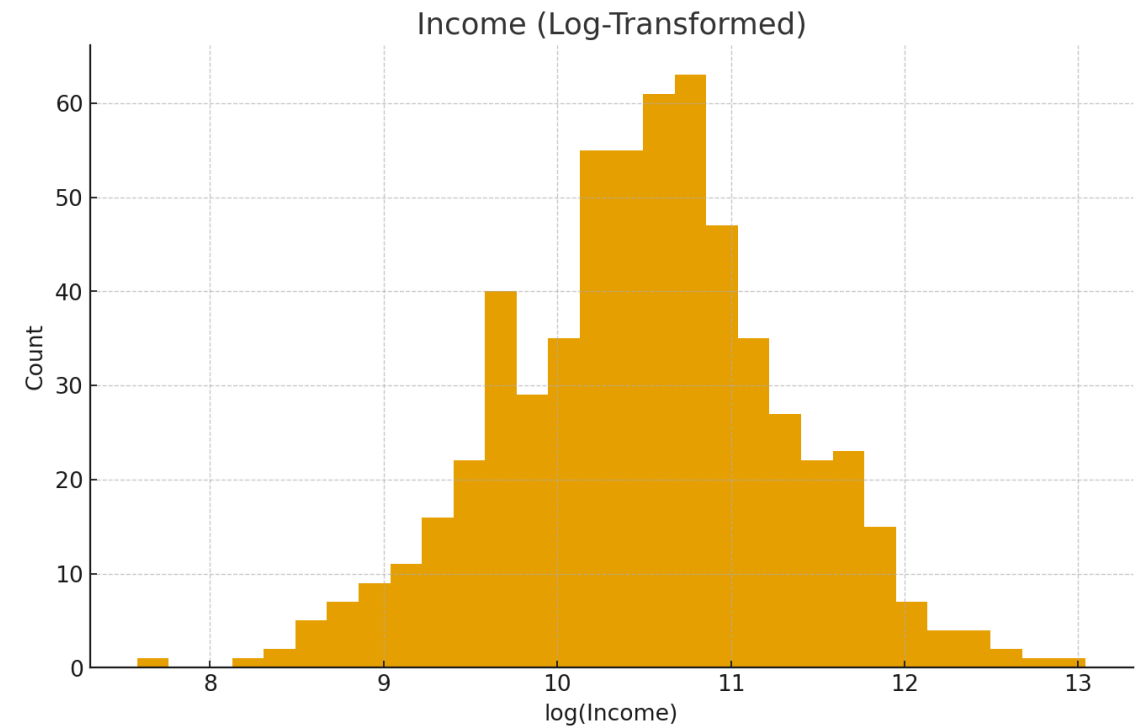


Figure: Example of log transformation on skewed data

Interactions Combine features to capture joint effects.

Example: $\text{BMI} = \text{weight} / (\text{height}^2)$.

Example: $\text{Age} \times \text{Exercise Frequency}$.

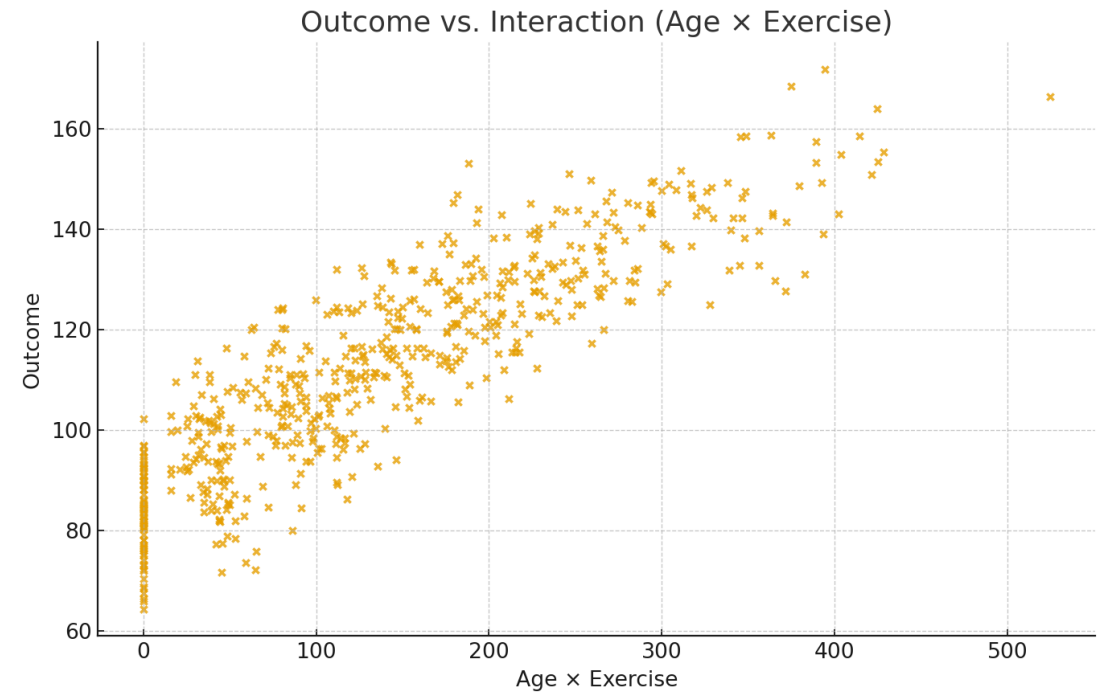


Figure: Example of interaction features

Feature selection chooses the most useful predictors

- More features $\neq$ always better
- Too many features can cause overfitting and slow training.
- **Feature selection:** Identifying the most informative subset
- Approaches include:

Filter methods

Correlation, variance threshold

Wrapper methods

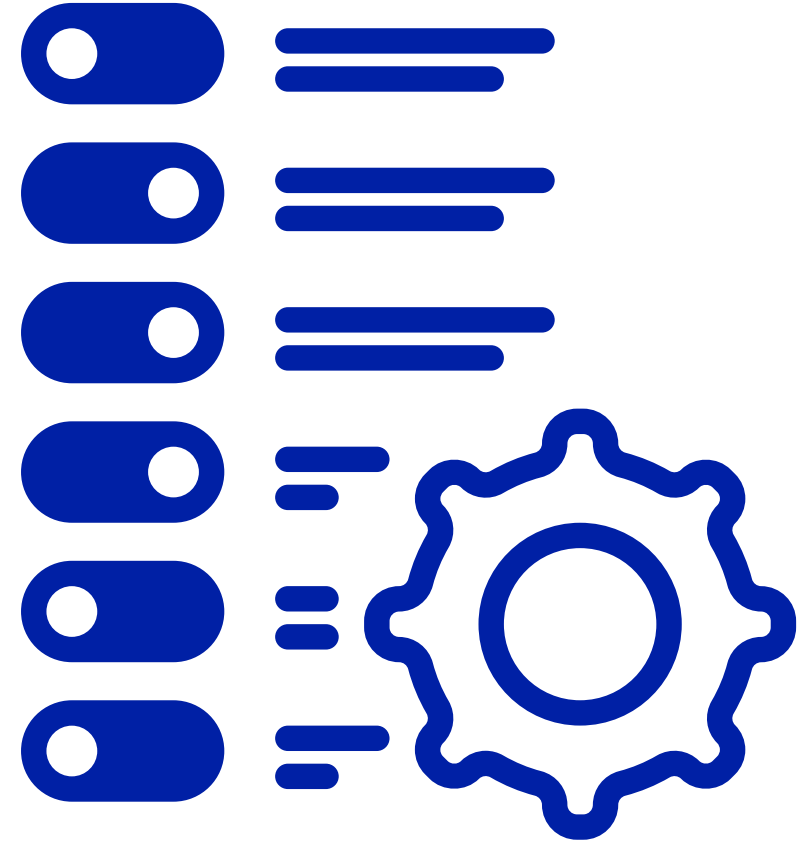
Forward/backward selection

Embedded methods

Regularization (e.g., Lasso),
decision tree importance

A small set of well-designed features often explains most predictive power

- Often, 20% of the features explain 80% of predictive performance.
- Good feature design can outperform sophisticated algorithms applied to poor data.
- Thoughtful feature engineering is more than just best fit selection.

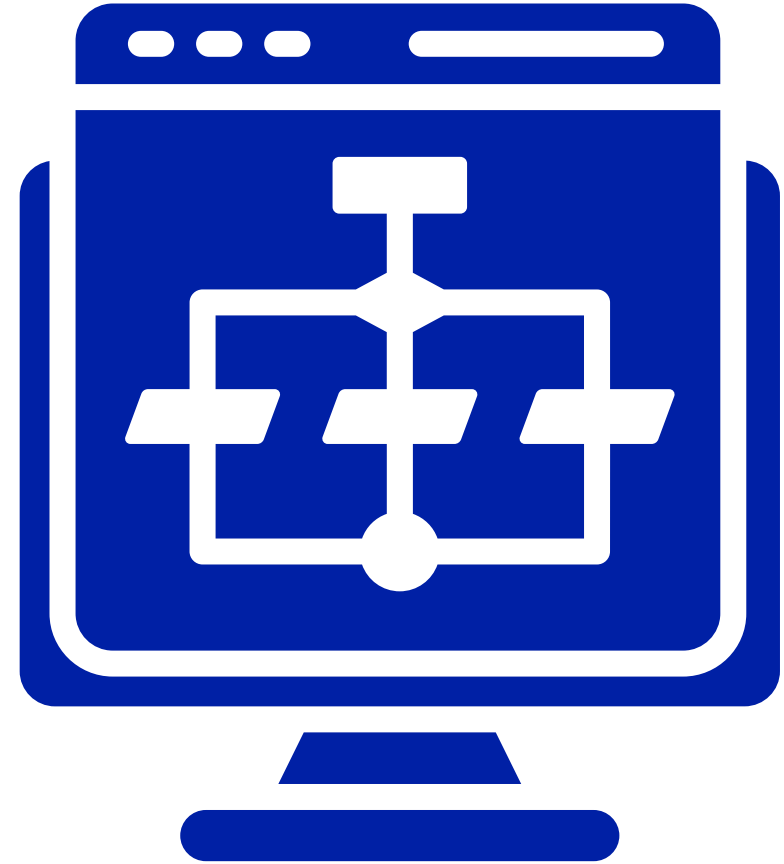




6. Evaluating Model Performance (Preview)

Evaluation is about generalization, not just training accuracy

- Training accuracy alone is not enough.
- We need metrics that reflect **how well the model generalizes**.
- Different tasks require different evaluation metrics.



Classification models are judged with metrics like accuracy, precision, recall, and F1-score



Balancing these different metrics can be tricky.

Figure: Confusion matrix showing
TP, FP, TN, FN

Accuracy: Proportion of correct predictions

Formula

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Use

Works when classes are balanced; can mislead on imbalanced data.

Figure: Confusion matrix showing
TP, FP, TN, FN

Precision: Of predicted positives, the fraction that are truly positive

Formula

$$\text{Precision} = \frac{TP}{TP + FP}$$

Use

Prioritize when false positives are costly (e.g., flagging legitimate emails as spam).

Figure: Confusion matrix showing TP, FP, TN, FN

Recall: Of actual positives, the fraction correctly identified

Formula

$$\text{Recall} = \frac{TP}{TP + FN}$$

Use

Prioritize when missing positives is costly (e.g., fraud or disease screening).

Figure: Confusion matrix showing TP, FP, TN, FN

F1-score: Harmonic mean of precision and recall

Formula

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Equivalently: $F1 = \frac{2TP}{2TP + FP + FN}$

Use

Balances precision and recall; useful with class imbalance.

Figure: Confusion matrix showing TP, FP, TN, FN

Regression models are judged with metrics like MSE, RMSE, and R^2

MSE (Mean Squared Error)

Average squared difference between predictions and actual values

RMSE (Root MSE)

Square root of MSE, keeps units of the target

R^2 (Coefficient of Determination)

Proportion of variance explained
(0 = none, 1 = perfect)

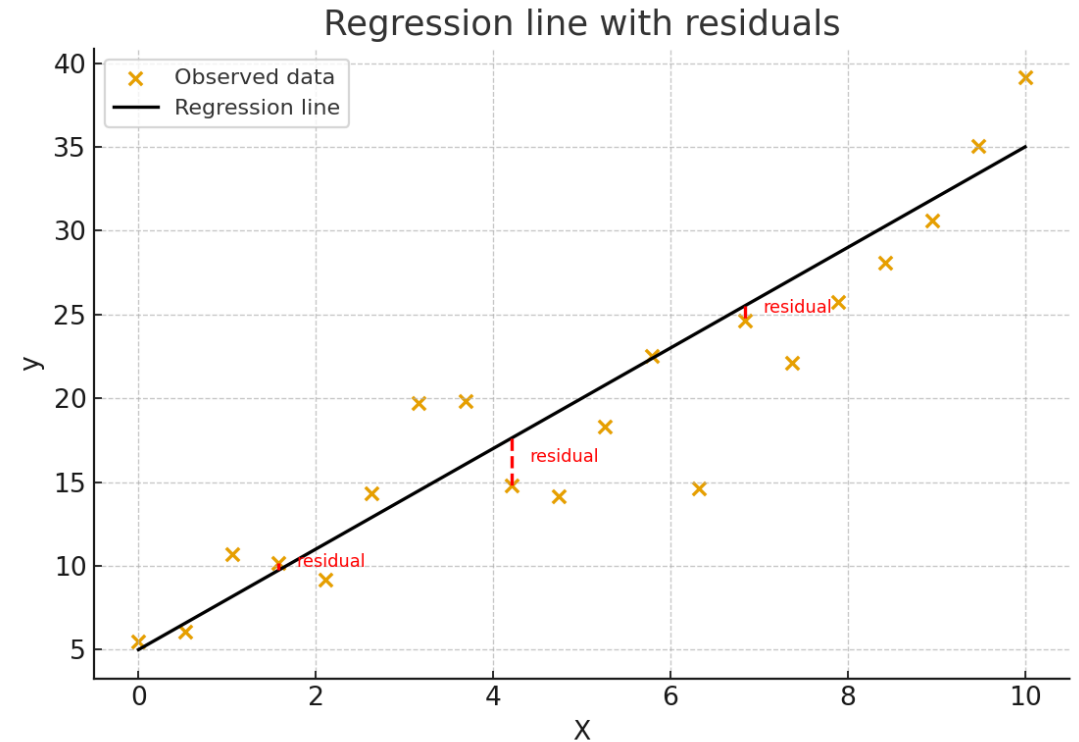


Figure: Residual errors plotted on regression line

The right evaluation metric depends on the context

- **Fraud detection** → Prioritize recall (*catch as many frauds as possible*).
- **Spam filters** → Prioritize precision (*avoid flagging legitimate emails*).
- **House price prediction** → RMSE is useful (*interpretable in dollars*).





Hands-On Exercise:
Apply the **ML pipeline** to a small dataset



References



References

A decorative vertical bar on the left side of the slide, composed of many thin, horizontal, light blue lines stacked vertically.

Nelson, B. (2007, July 19). *Checkers computer becomes invincible*. NBC News.
<https://www.nbcnews.com/id/wbna19839044>